

RTI Assistant

A Retrieval-Augmented Generation (RAG) Chatbot
for Indian Government RTI Services

Statistical Methods in AI (SMAI)

Assignment 3 — Theme T10.3

Team Name: The3

Shada Praneeth Reddy	2025204006	shadapraneeth.reddy@students.iiit.ac.in
Nagam Chandrakanth Reddy	2025204003	nagam.reddy@students.iiit.ac.in
Pittada Sai Harsha Vardhan	2025204021	pittada.s@students.iiit.ac.in

GitHub Repository: [View Repository](#)

Demo Video (YouTube): [Watch Demo](#)

Demo Video (Drive): [Open Drive Link](#)

May 6, 2026

Contents

1	Introduction	1
2	Dataset and Knowledge Base	2
2.1	Knowledge Base Documents	2
2.2	Corpus Statistics	3
2.3	Data Quality Notes	3
3	Methodology	3
3.1	Ingestion Pipeline (ingest.py)	3
3.2	Query Pipeline	4
3.2.1	History-Aware Query Condensation	4
3.2.2	MMR Retrieval	4
3.2.3	Prompt Engineering	4
3.2.4	Retry Logic	5
3.2.5	Streaming Output	5
4	System Architecture	5
4.1	Architecture Diagram	5
4.2	Sequence Diagram	6
5	Hindi Language Support	6
6	Results and Evaluation	7
6.1	Sample Interactions	7
6.2	Application Pages	8
6.3	Chunk Size Ablation Study	9
6.4	Retrieval Strategy Comparison	9
7	Limitations	9
8	Future Enhancements	10
9	Conclusion	10
10	References	10

1. Introduction

Indian government services, particularly the Right to Information (RTI) Act, are backed by extensive official PDF documentation covering legal definitions, procedural rules, online filing steps, and FAQs. For a typical citizen, locating a specific fact such as the application fee, response deadline, or appeal process requires manually scanning multiple dense documents, which is impractical.

We address this with a Retrieval-Augmented Generation (RAG) chatbot that:

1. Indexes official RTI PDFs into a local vector database at ingestion time.
2. At query time, retrieves the most semantically relevant passages and supplies them as grounded context to a large language model (LLM).

- Generates a concise, verifiable answer with source citations, referencing only retrieved passages and not the model's parametric knowledge.

This design eliminates hallucination risk, makes every answer auditable, and requires no model training or fine-tuning. The system is packaged as a Streamlit web application.

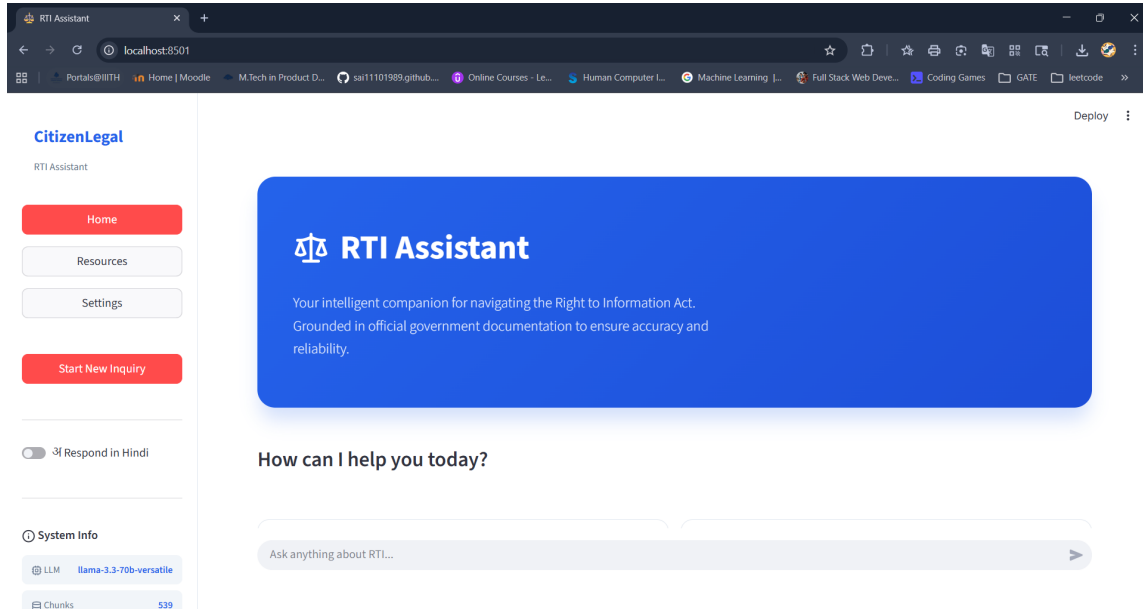


Figure 1: RTI Assistant Home Page

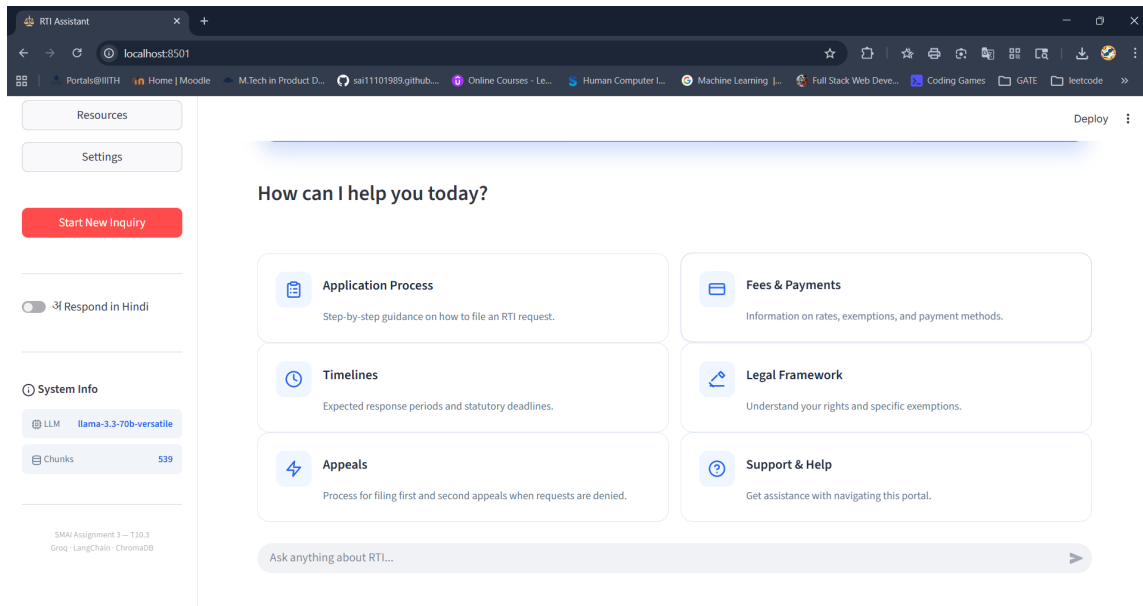


Figure 2: Home Page Displaying Topic Cards

2. Dataset and Knowledge Base

2.1 Knowledge Base Documents

The system uses official RTI-related documents obtained from the Government of India RTI Portal.

Table 1: Knowledge Base Documents

File Name	Description	Type
RTI Act 2005 (Amended)-English Version.pdf	Full text of the RTI Act, 2005	Act
RTI-Act.pdf	Alternate copy of RTI Act	Act
RTI_Rules_2019_hindi_english.pdf	RTI Rules (Bilingual)	Rules
RTI FAQ English.pdf	Frequently Asked Questions	FAQ
RTI Online User Manual English.pdf	User manual for RTI online portal	Manual

2.2 Corpus Statistics

After ingestion: 88 raw pages produce 539 text chunks (average 536 characters per chunk).

Table 2: Corpus Statistics

Document Type	Chunks
Act	280
Manual	149
Rules	65
FAQ	45

2.3 Data Quality Notes

PDFs are parsed using PyPDF via LangChain’s `PyPDFDirectoryLoader`. The bilingual Rules PDF retains Hindi text, which benefits Hindi-mode retrieval. One PDF (`RTI-Act.pdf`) triggers a non-standard header warning but is parsed correctly after fallback recovery by the library.

3. Methodology

3.1 Ingestion Pipeline (`ingest.py`)

The offline pipeline runs once and produces the persistent vector index.

RTI PDFs → PyPDFLoader → SmartChunker → MetadataEnricher → MiniLM Encoder → ChromaDB

Step 1: Document Loading. `PyPDFDirectoryLoader` reads all pdf files in the data directory, extracting page text and metadata such as source filepath and page number.

Step 2: Smart Chunking. `RecursiveCharacterTextSplitter` uses a custom separator hierarchy that respects RTI legal structure before falling back to finer splits.

Section N / Chapter IV / Rule N -> paragraph -> line -> sentence -> word -> character

Parameters:

- `chunk_size = 700`
- `chunk_overlap = 150`

- `add_start_index = True`

Why 700 characters? The `all-MiniLM-L6-v2` model has a 512-token context window corresponding to approximately 700–900 English characters. Staying within this range avoids embedding truncation while improving retrieval precision.

Step 3: Metadata Enrichment. Each chunk receives:

- `doc_type`: classified from filename (Act, Rules, FAQ, Manual)
- `chunk_id`: deterministic SHA256-based identifier for idempotent ingestion

Step 4: Embedding and Persistence. Chunks are embedded using `all-MiniLM-L6-v2` (384-dimensional local embedding model) and stored in ChromaDB with persistent storage enabled.

3.2 Query Pipeline

The online inference pipeline processes user queries and generates context-aware answers.

User Query → History-Aware Condenser → MMR Retriever → Prompt Builder →
Groq LLM → Streamlit UI

3.2.1 History-Aware Query Condensation

Conversation history is used to rewrite follow-up queries into self-contained semantic search prompts.

Example:

- User Follow-up: *“What about BPL applicants?”*
- Condensed Query: *“RTI application fee exemption for BPL applicants”*

This significantly improves retrieval precision.

3.2.2 MMR Retrieval

Maximal Marginal Relevance (MMR) is used to improve diversity among retrieved chunks and avoid near-duplicate passages.

Configuration:

- `fetch_k = 20`
- `k = 6`
- `lambda = 0.7`

3.2.3 Prompt Engineering

The system prompt strictly instructs the model to:

- Answer only from retrieved context.
- Avoid hallucinations.
- Cite relevant sections and rules.
- Support Hindi responses when enabled.

3.2.4 Retry Logic

Tenacity-based exponential backoff handles temporary API failures and rate limits.

3.2.5 Streaming Output

Streaming responses improve perceived latency and user experience within the Streamlit application.

4. System Architecture

The application consists of three primary user-facing views:

Table 3: Application Views

View	Purpose
Home	Main chatbot interface with topic cards and interaction panel
Resources	PDF browsing and download interface
Settings	Language toggle and model configuration display

4.1 Architecture Diagram

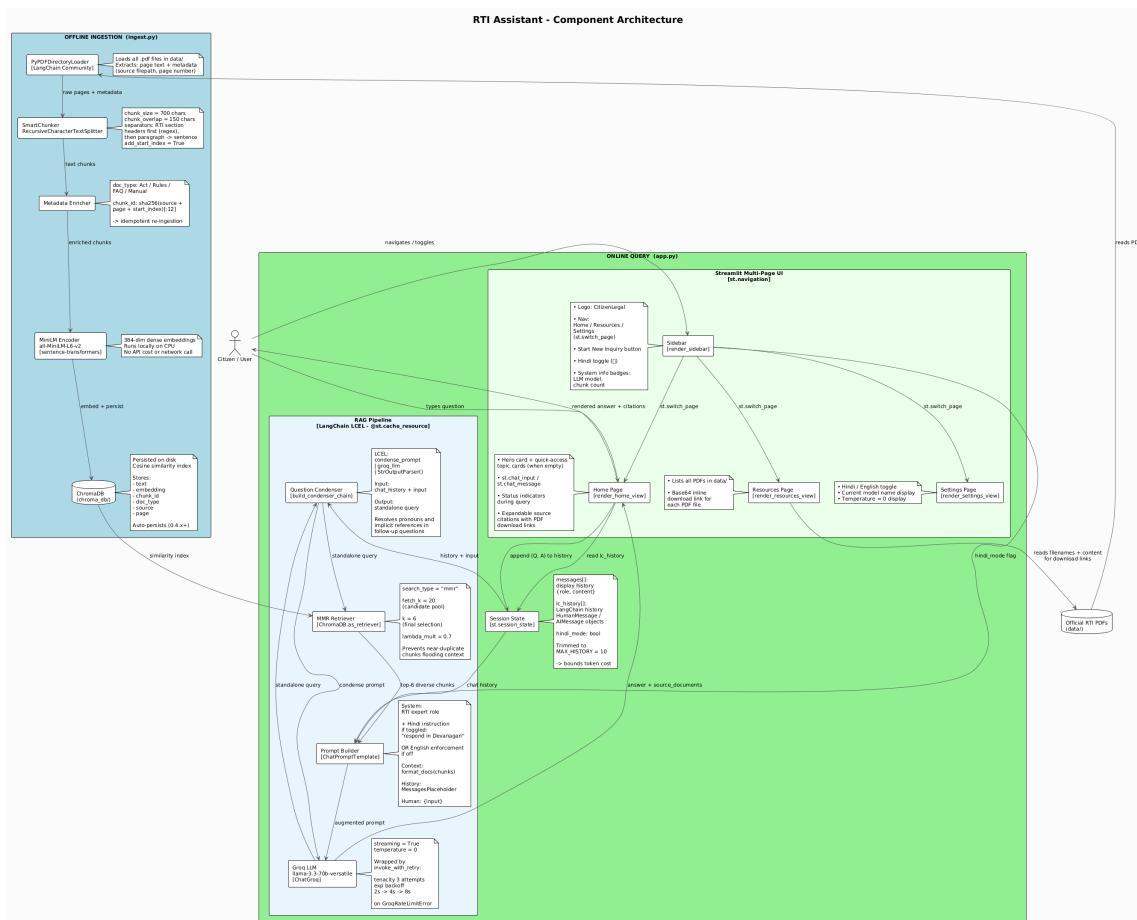


Figure 3: Component Architecture Diagram

4.2 Sequence Diagram

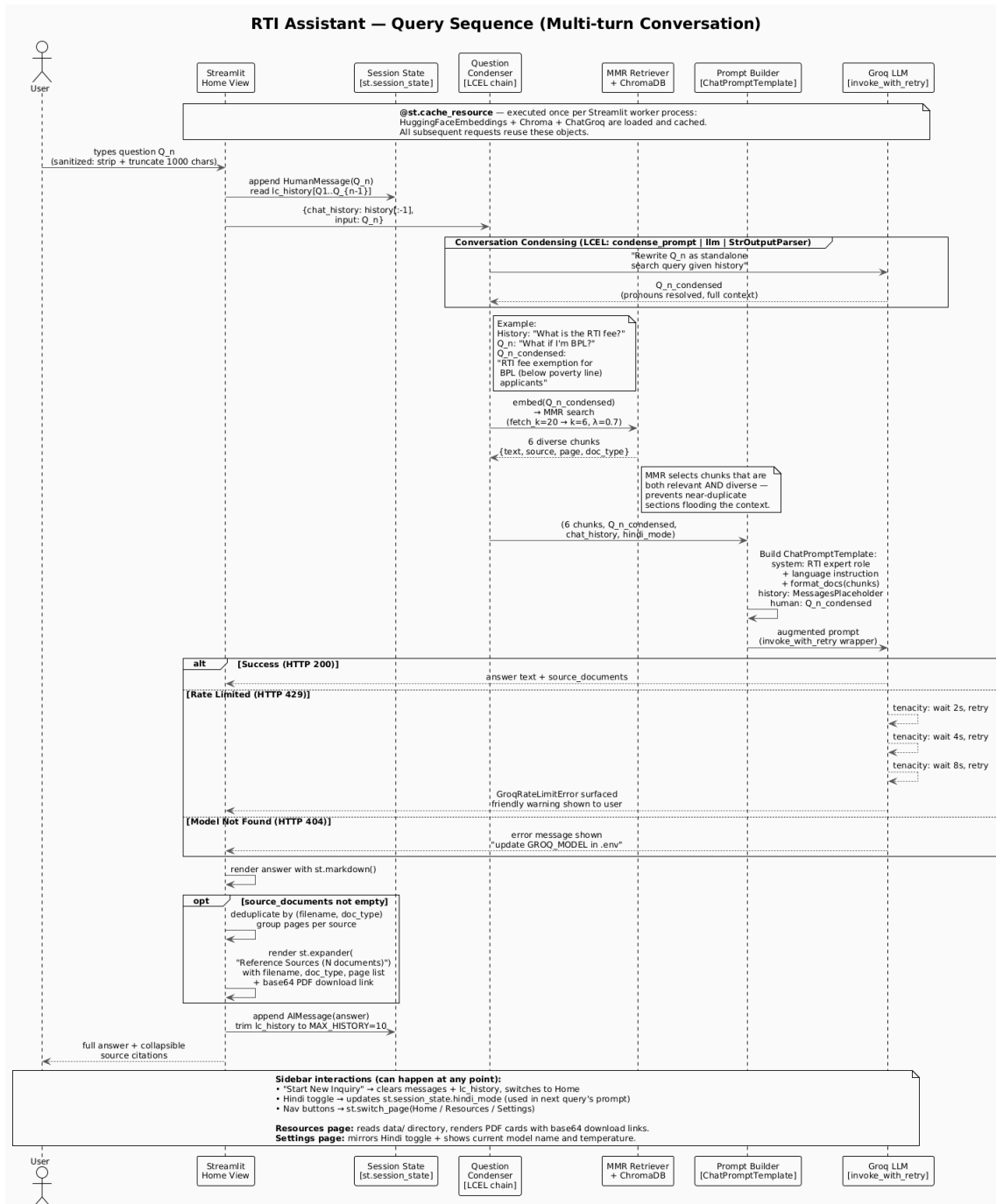


Figure 4: Query Processing Sequence Diagram

5. Hindi Language Support

The application supports bilingual interaction through a dedicated Hindi toggle.
When enabled:

- Responses are generated entirely in Hindi.
- Devanagari script is enforced.

- The chat interface dynamically updates placeholders.

This improves accessibility for native Hindi-speaking users.

6. Results and Evaluation

6.1 Sample Interactions

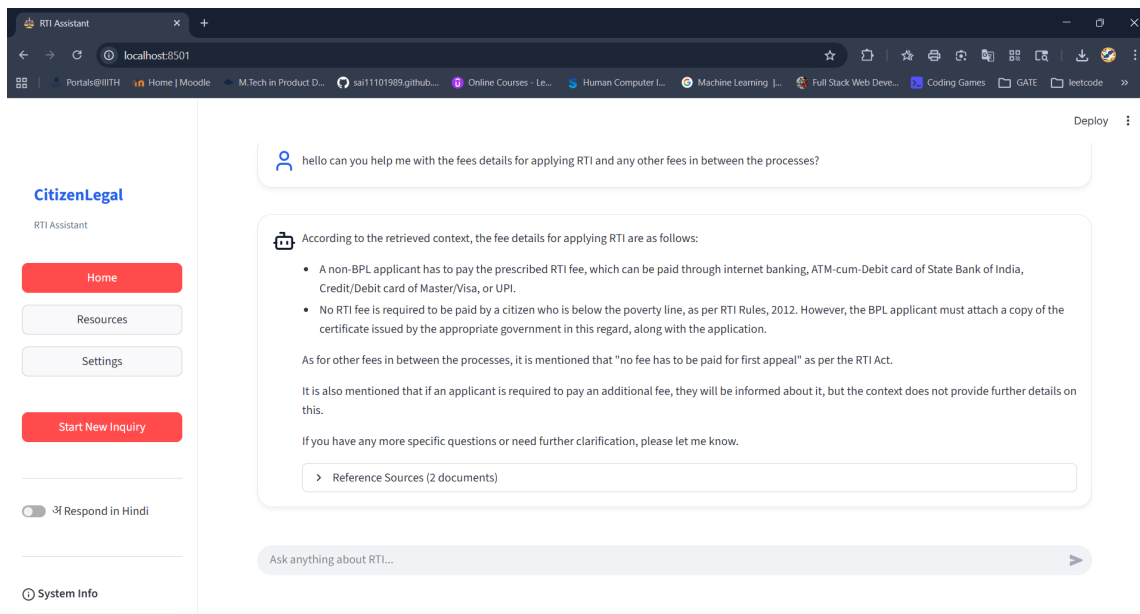


Figure 5: English Mode Question and Response

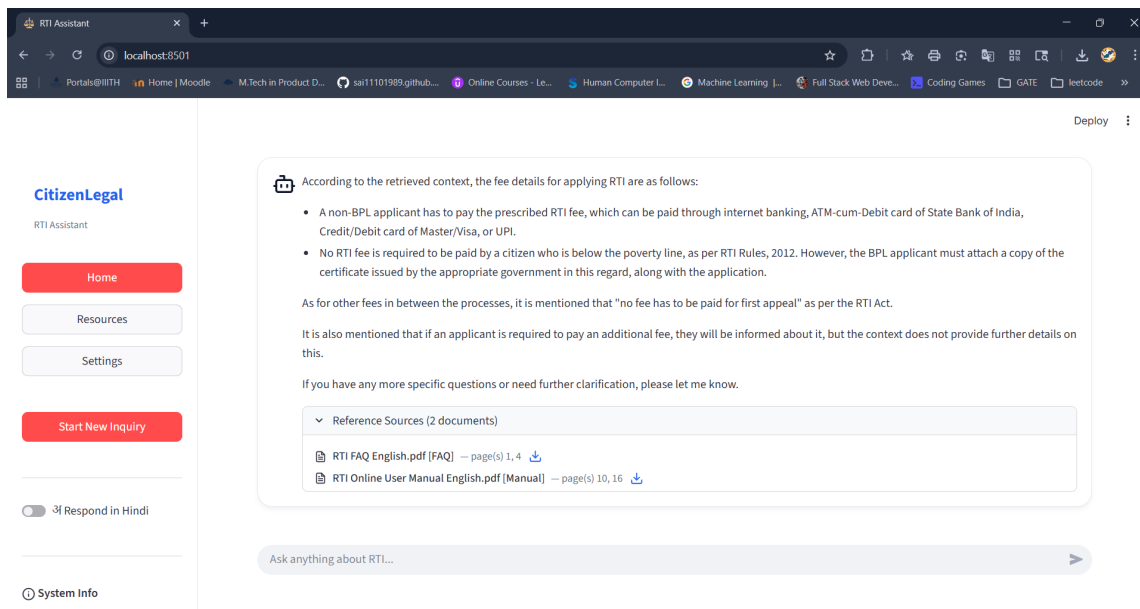


Figure 6: Citation Display Interface

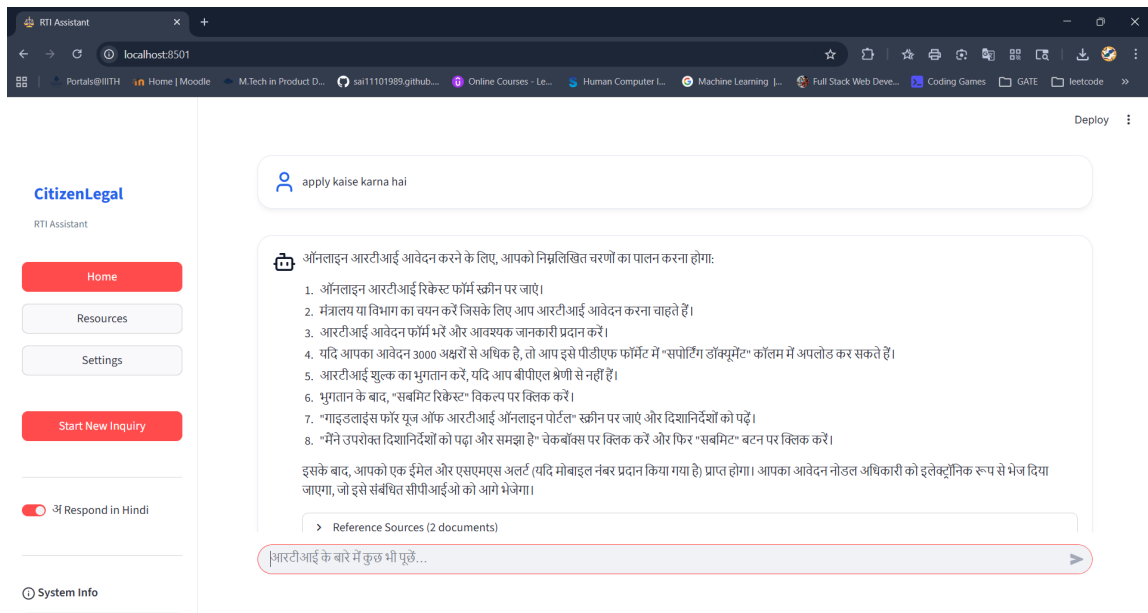


Figure 7: Hindi Mode Response

6.2 Application Pages

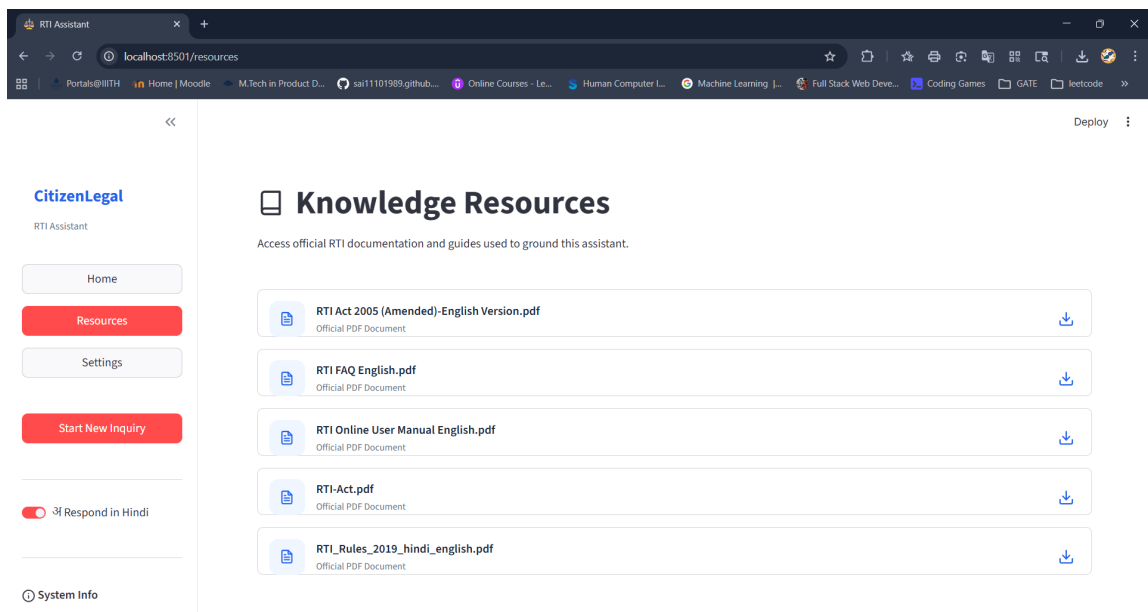


Figure 8: Resources Page

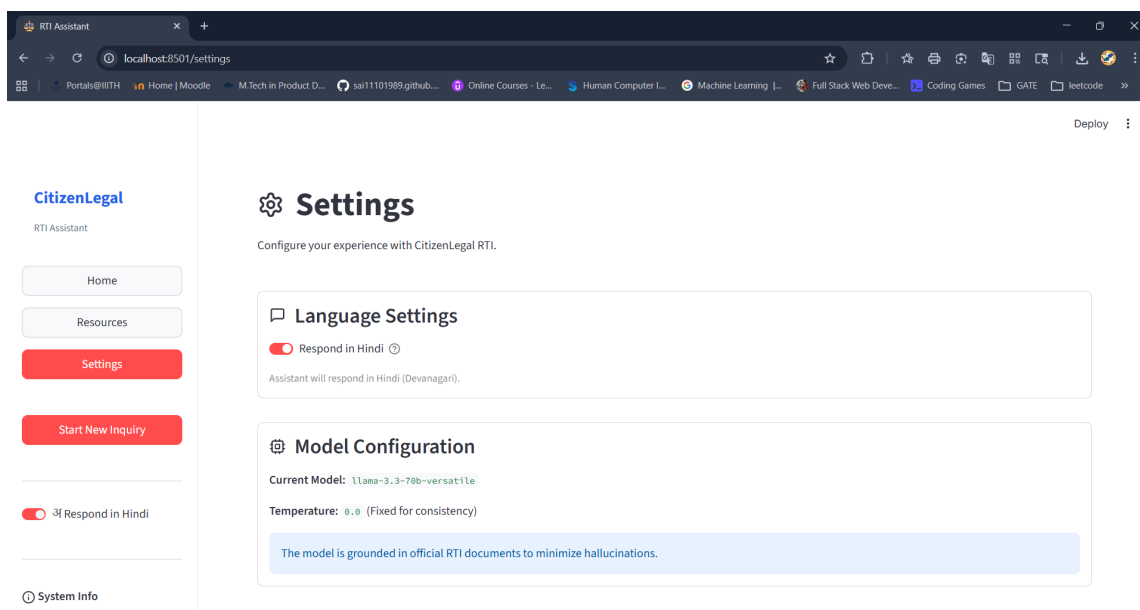


Figure 9: Settings Page

6.3 Chunk Size Ablation Study

Table 4: Effect of Chunk Size on Retrieval Quality

Chunk Size	Overlap	Chunks	Observation
1000	200	380	Large paragraphs reduce retrieval precision
700	150	539	Best balance between precision and context
400	100	900	Over-fragmentation and context loss

6.4 Retrieval Strategy Comparison

Table 5: Comparison of Retrieval Methods

Method	Context Diversity	Answer Quality
Top-k Cosine Retrieval	Low	Misses complementary information
MMR Retrieval	High	Produces more comprehensive answers

7. Limitations

1. PDF parsing loses structural alignment for complex tables.
2. The embedding model is primarily English-focused.
3. Knowledge base updates require re-ingestion.

4. Free-tier API limits restrict scalability.

8. Future Enhancements

Potential future improvements include:

- Multilingual embedding models.
- Automated document update pipelines.
- OCR-enhanced PDF understanding.
- Voice-based RTI interaction.
- Deployment using scalable cloud infrastructure.

9. Conclusion

This project demonstrates how Retrieval-Augmented Generation can be effectively applied to government information systems. By combining semantic retrieval, vector databases, and modern LLM-based response generation, the RTI Assistant provides citizens with a fast, transparent, and reliable way to access RTI-related information.

The system successfully achieves:

- Context-grounded answer generation
- Reduced hallucination risk
- Explainable and verifiable outputs
- Multilingual accessibility
- Efficient retrieval from official legal documents

The architecture is modular, scalable, and suitable for future expansion into broader e-governance domains.

10. References

1. Lewis, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.
2. Carbonell, J. and Goldstein, J. *The Use of MMR for Reordering Documents and Producing Summaries*. SIGIR 1998.
3. Government of India. *The Right to Information Act, 2005*. <https://rti.gov.in>
4. Government of India. *The Right to Information (Amendment) Rules, 2019*.